Master Report

Nov 1, 2026 · @Hardik

Task 1: Segmentation with a pretrained model

Approaches

1. Detect, then segment
   --> An open-vocabulary detector finds boxes from the text prompt "egg", and SAM 3 turns each box into a mask.
   --> I tried three detectors: Grounding DINO, OWLv2 and YOLO-World.
   --> I also tried a fourth variant that merges the boxes of all three detectors (Weighted Box Fusion) before passing them to SAM 3.
2. Segment directly from text
   --> SAM 3 can take the word "egg" itself and return a mask for every egg, with no separate detector.
3. That gave five setups. I ran all of them on all 110 images on a cloud GPU (Modal, H100).

Comparison

1. For every image I compared the setups with each other: the number of eggs each found, and how much their masks overlap (IoU). Images where they disagreed were flagged and checked by eye.
2. Results per setup
   --> Grounding DINO + SAM 3: found eggs in all 110 images, 141 eggs in total.
   --> OWLv2 + SAM 3: found no egg in 22 images, 120 eggs in total.
   --> YOLO-World + SAM 3: found no egg in 12 images, 136 eggs in total.
   --> Box fusion + SAM 3: found eggs in all 110 images, 142 eggs in total.
   --> SAM 3 text: found no egg in 3 images, 138 eggs in total.
3. OWLv2 and YOLO-World were clearly the weakest: they missed every egg in 22 and 12 images respectively. They also agreed with the other setups only about 0.77 to 0.83 IoU.
4. SAM 3 text and Grounding DINO + SAM 3 agreed closely (mean IoU 0.96), and differed on only 7 images.
5. In two of those 7 images, the Grounding DINO-based setups counted part of a printed label next to the egg as a second egg, while SAM 3 text correctly found one.

Final choice

1. I used SAM 3 with the text prompt "egg" as the ground truth.
   --> It gave the cleanest masks and the fewest false eggs.
   --> It needs no extra detector or box threshold.
2. In the 3 images where it found nothing (eggs in boiling water and a white egg tray), I used Grounding DINO + SAM 3 instead. The final ground truth covers all 110 images and 141 eggs.
3. One known error remains. In image 117, a bar crosses the egg and the mask covers only half of it. I left it unchanged and kept it in mind when reading the results of the later tasks.

Where to find it

1. Code
   --> scripts/run_task1.py (all five setups)
   --> scripts/make_gt.py (builds the ground truth)
   --> scripts/task1_compare.py (comparison tables)
2. Experiment logs and results (in outputs/task1/)
   --> method_comparison.csv and pairwise_iou.csv: the comparison numbers
   --> summary.csv: per-image counts and overlaps
   --> review_these.txt: the flagged images
   --> comparison/: all five setups side by side for every image
   --> gt_review/: the 7 disagreements
3. Final masks: data/gt_masks/

Task 2: Segmentation with classical image processing

Splitting the data

1. Leakage source: all 110 images come from different source files, so there are no direct duplicates. The real leakage risk is that many photos are several shots of the same egg on the same background.
2. Sessions: I grouped the images into 25 photo sessions. Candidate matches came from image-similarity checks (SIFT feature matching, perceptual hashing and colour histograms), and I confirmed each session by eye.
3. Split method: I split with StratifiedGroupKFold, so each session goes entirely to dev or entirely to eval. It is also balanced by scene type.
4. Result: 87 dev / 23 eval images, with no session in both.

5. Check: eval images are no more similar to dev images than dev images are to each other (outputs/task2/split/leakage_check.png).
6. Cost: eval has no ash, plant or keyboard scenes.
7. Rule: all development used the dev split only. The eval split was used once, in Task 3.

EDA: what the data told me (dev only, plots in outputs/task2_1/eda/)

1. Egg location (B)
   --> Finding: eggs sit near the centre, and under 2% touch the image border.
   --> So: prefer central blobs and drop blobs that touch the border.
2. Colour channels (C, D)
   --> Finding: brightness separates egg from background best; hue is useless because the eggs are white.
   --> So: work on grayscale.
3. Egg vs surroundings (E)
   --> Finding: the egg is brighter than the ring around it in 86 of 87 images, but it isn't always the brightest thing in the image.
   --> So: a single global threshold won't always work.
4. Image clusters (F)
   --> Finding: k-means finds three kinds of background: dark/colourful, bright/plain and grey/cluttered. Otsu works on the first and fails on the other two.
   --> So: the threshold has to adapt per image.
5. Otsu vs best threshold (H)
   --> Finding: Otsu gives 0.33 IoU, and even the best possible threshold per image only gives 0.48.
   --> So: thresholding alone is not enough; the masks need cleaning and refining.
6. Blob shapes (I)
   --> Finding: egg blobs are large and solid (solidity about 0.97). Background blobs are small, irregular and often on the border.
   --> So: simple filters on area, solidity and border contact can pick out the eggs.
7. Edges (J)
   --> Finding: the egg outline is often a visible edge, but the backgrounds are full of edges too.
   --> So: edges can refine a mask but can't find the egg on their own.
8. The plan from the EDA: threshold the grayscale image, keep the blobs that look like eggs, then refine them.

Building the algorithm step by step

1. How each trial was judged

--> Each trial changed one thing.
--> I measured the dev IoU change, with a bootstrap confidence interval.
--> I counted how many images got better or worse.
--> A change was kept only if it clearly helped.

2. The log has 23 trials plus a simplification round, including the failures.
3. Step 1, baseline
   --> Otsu on grayscale scored 0.335.
   --> I added clean-up (opening, hole filling) and the blob filters. This made the output precise, but it exposed the real problem: on bright or cluttered backgrounds the egg merges with the background and gets rejected.
4. Step 2, adapt the threshold
   --> If Otsu finds no egg-like blob, try multi-Otsu with 3 classes, then with 4.
   --> This matches the three background types from the EDA, and the score rose to 0.465.
5. Step 3, recover the whole egg
   --> A raised threshold catches only the lit half of the egg.
   --> Refining each blob with GrabCut, and letting it grow slightly, fixed this. The score rose to 0.56.
6. Step 4, fall back on location
   --> When no blob looks like an egg, run GrabCut on the central part of the image, since that's where eggs usually are.
   --> The score rose to 0.649.
7. What didn't work (all logged)
   --> k-means instead of thresholding
   --> splitting merged blobs with watershed
   --> bigger morphological openings
   --> voting ensembles
   --> choosing between two pipelines per image
8. Simplifying
   --> I removed one step at a time to see which steps actually earned their place.
   --> The adaptive threshold step and a special growth rule could go with no loss.
   --> The simpler version scored slightly higher: 0.663.

Final algorithm

1. Convert to grayscale.
2. Threshold cascade: Otsu, then multi-Otsu (3 classes), then multi-Otsu (4 classes), stopping at the first level that gives an egg-like blob.
3. Clean up with opening (radius 2) and hole filling.

4. Keep blobs with area 1 to 40% of the image, solidity at least 0.85, and not touching the border.
5. Pick the most central blob, plus any other blob at least half its size.
6. Refine each egg with GrabCut.
7. If nothing was found, run GrabCut on the central 60% of the image.

Dev results (87 images)

1. Mean IoU 0.663, median 0.751, Dice 0.737.
2. Correct egg count in 83% of images.
3. 0.45 s per image on one CPU core.
4. Works best on dark or colourful backgrounds; struggles on white surfaces and hands, where the egg blends into the background.

Where to find everything

1. Split: outputs/task2/split/ (split_summary.json, leakage_check.png)
2. EDA plots and numbers: outputs/task2_1/eda/ (files A to J, eda_summary.json)
3. Experiment log: outputs/task2_1/experiments/experiment_log.csv. It records what changed, why, the result and the decision for every trial.
4. Progress over the trials: outputs/task2_1/figures/progression.png, cluster_progression.png
5. Final results on dev: outputs/task2_1/final/dev/ (masks, overlays, metrics.csv, summary.json); successes and failures in outputs/task2_1/figures/final_dev_examples.jpg
6. Code
   --> scripts/make_split.py
   --> scripts/task21_eda.py
   --> scripts/task21_experiments.py
   --> src/eggseg/simple.py (the final algorithm)
   --> scripts/run_task2.py

Task 3: Evaluation and analysis

Setup

1. Ground truth: the Task 1 masks (SAM 3 text, with Grounding DINO + SAM 3 on 3 images).
2. Predictions: the final Task 2 algorithm, frozen before this step.

3. Data: the 23-image eval split, used once. Dev results are shown alongside for comparison.
4. Known ground-truth error: image 117 (a dev image) is only half covered. I left it as is, because removing it changes the dev score only from 0.663 to 0.668.

Metrics used

1. IoU and Dice: how well the predicted mask overlaps the true mask.
2. Precision: how much of the predicted area is actually egg. Low precision means the mask spills onto the background.
3. Recall: how much of the egg was covered. Low recall means part of the egg is missing.
4. Egg count accuracy: whether the method found the right number of eggs.
5. Per-egg matching: an egg counts as found if its predicted mask overlaps it with IoU of at least 0.5.
6. Baseline: plain Otsu thresholding, for reference.

Results on eval (23 images)

1. Mean IoU 0.56 (95% CI 0.44 to 0.68), compared with 0.66 on dev.
2. Median IoU 0.61; Dice 0.67.
3. Precision 0.66; recall 0.81.
4. IoU of at least 0.5 on 61% of images; at least 0.8 on 26%.
5. Correct egg count on 83% of images.
6. Eggs found: 13 of 30 (43%), plus 10 false eggs.
7. Plain Otsu scores 0.32, so the method is about 1.8 times better than the baseline.

Why eval is lower than dev

1. Different scene mix
   --> Eval happens to contain more of the hardest scenes (hands and white surfaces).
   --> It has none of the easiest (ash and plants).
   --> Re-weighting the dev results to eval's scene mix already gives 0.60, so most of the drop is the scene mix, not overfitting.
2. Scores by scene on eval
   --> fabric 0.74, coloured plastic 0.65, white surface 0.63
   --> hand 0.43, stone counter 0.46

What kind of mistakes it makes

1. Outcome counts: 6 images good (IoU at least 0.8), 8 usable (0.5 to 0.8), 5 spill into the background, 4 wrong object or missed.
2. The main error is too much, not too little
   --> Recall is high (0.81), so the egg is usually fully covered.
   --> But the mask often spreads onto neighbouring areas that look similar.

Visual review (all 23 eval images checked by eye)

1. Works well
   --> Eggs on coloured or darker plain surfaces (#21, #34, #115, #16, #52).
   --> The mask follows the real egg outline, including its shaded side.
2. Spills onto egg-coloured surroundings (#1, #2, #20, #102 on hands; #149, #150 on tiles; #32, #40 on white paper)
   --> The egg is found, but the mask leaks into the palm, wall or shadow.
3. Half an egg (#30, #35)
   --> One side is in shade, and only the lit half is kept.
4. Touching eggs (#142, #144, #148)
   --> Three eggs pressed together read as one big blob.
   --> That blob is either accepted as one egg or rejected entirely.
5. Wrong bright object (#22)
   --> A large round printed label is picked instead of the small egg beside it.
6. Dark egg (#24)
   --> The egg is backlit against a window, so it is darker than its surroundings.
   --> This breaks the method's basic assumption that eggs are bright.
7. Fallback box (#44)
   --> Nothing passed the filters, so the centre fallback produced a boxy mask.
8. Compared with SAM 3
   --> SAM 3's masks look correct on all of these images.
   --> Every failure above needs knowledge of what an egg is, such as its shape, that eggs can touch, or that a label is not an egg.
   --> Brightness and colour alone can't provide that.

How it could be improved

1. Shape check on the final mask: reject or trim masks that are far from an egg-like ellipse. This targets the spills.
2. Split touching eggs using egg size and shape, instead of treating one big blob as a single egg.

3. Skin-colour filter (low saturation only) for scenes with hands. This already worked well in Task 4.
4. A larger minimum blob size (3% instead of 1%), so small bright patches can't win. This was tested in Task 2; it helped, but the result was not yet conclusive.
5. Handle dark-on-bright eggs by also looking for blobs darker than their surroundings.

Where to find everything

1. Code: scripts/task3_eval.py. Run scripts/run_task2.py --split eval first.
2. Numbers
   --> outputs/task3/summary.json: headline results
   --> per_image.csv: every image's scores and outcome
   --> per_scene.csv: results by scene
3. Plots
   --> outputs/task3/dev_vs_eval.png: dev vs eval
   --> by_scene.png: by scene
   --> error_types.png: precision vs recall, showing the spill pattern
4. Wrongly segmented samples: outputs/task3/eval_gallery.jpg, all eval images from worst to best. Red is the prediction, green the SAM 3 outline.

Task 4: Video tracking

1. Segmentation: each frame is segmented with the final Task 2 algorithm, with small changes for video. These remove false detections on title cards, hands and background patches.
2. Tracking: each detected egg is linked across frames with a Kalman filter and Hungarian matching. This is the standard approach behind widely used trackers such as SORT.
3. Why a Kalman filter
   --> It predicts where each egg should be in the next frame from its position and speed.
   --> This keeps an egg's ID through short gaps, such as a finger passing over it or a missed detection.
   --> It also smooths out jitter in the detected positions.
4. Why Hungarian matching
   --> Each frame, it pairs predicted positions with new detections so that the total distance is as small as possible.
   --> Each egg gets at most one detection, so the two eggs don't swap IDs when they're close together.
5. Why this combination
   --> It is simple, fast and needs no training.

--> It suits this video: a few objects moving smoothly on a fixed background.

6. Result
   --> While the eggs are on the table, each keeps its own ID, with no swaps and no false tracks.
   --> An egg gets a new ID only when the hand picks it up and puts it down elsewhere, because it is hidden for too long to follow.
7. Task 4 tracked video: https://drive.google.com/file/d/1hbDwUgPEwV0BYkP3mvo0WxX8rCifEjkQ/view?usp=sharing
8. Code: scripts/run_task4.py, src/eggseg/track.py

AI tools

1. I used Claude (chat) and Claude Code as coding assistants throughout the project.
2. What they helped with
   --> Writing and debugging the code for all four tasks, and setting up the cloud GPU run for Task 1.
   --> Running the experiments and producing the plots and tables.
3. What I decided myself
   --> Which pretrained models to compare, and SAM 3 text as the ground truth.
   --> Rejecting my first Task 2 pipeline as too ad hoc, and rebuilding it from simple EDA and standard techniques.
   --> Using the eval split only once, after the algorithm was frozen.
   --> Using a Kalman filter tracker for the video.
4. The full log, by date, is in docs/ai_usage.md.

Setup

1. Instructions to set up the repo and run every experiment are in README.md.